

Universidad San Carlos de Guatemala  
Facultad de ingeniería.  
Ingeniería en ciencias y sistemas



**FIUSAC**  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



## Parser JS

**PONDERACIÓN: 25 pts**

 **Tiempo estimado: 20 hrs**

## Índice

|      |                                                         |           |
|------|---------------------------------------------------------|-----------|
| 1.   | MARCO FORMATIVO .....                                   | 3         |
| 1.1. | Valor.....                                              | 3         |
| 1.2. | Competencia(s).....                                     | 3         |
| 1.3. | Objetivo SMART .....                                    | 3         |
| 2.   | Resumen Ejecutivo .....                                 | 4         |
| 3.   | Enunciado del Proyecto .....                            | 4         |
| 3.1  | Descripción del problema a resolver.....                | 4         |
| 3.2  | Alcance del proyecto – Sintaxis del Lenguaje.....       | 5         |
|      | <b>Requerimientos Técnicos.....</b>                     | <b>11</b> |
| 3.3  | Entregables .....                                       | 12        |
| 4.   | Material de apoyo .....                                 | 12        |
| 6.   | Recursos y herramientas a utilizar.....                 | 13        |
| 7.   | Desarrollo de Habilidades Blandas.....                  | 14        |
| 8.   | Cronograma .....                                        | 14        |
| 9.   | Rúbrica de Calificación .....                           | 15        |
|      | <b>9.1 Requisitos para Optar a la Calificación.....</b> | <b>15</b> |

## 1. MARCO FORMATIVO

### 1.1. Valor

| Nombre del valor | ¿Cómo se aplica en tu laboratorio?                                                                                                                                                                                                                                                                                                                                |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Excelencia       | Los estudiantes demostrarán excelencia técnica al implementar un analizador léxico funcional y preciso, aplicando los principios de teoría de autómatas y lenguajes formales con rigor profesional. La calidad de la solución desarrollada, junto con su documentación y la interfaz de usuario, reflejará el compromiso con los más altos estándares académicos. |

### 1.2. Competencia(s)

| Tipo de Competencia    |                                                                                                                                                                                                                                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Competencia General    | Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software.                                                                                                                                |
| Competencia Específica | Construir analizadores léxicos aplicando conocimientos de autómatas finitos deterministas (AFD), programación orientada a objetos, recursividad y paradigmas de programación imperativos, con el fin de definir los estados del AFD, procesar archivos de entrada con una sintaxis establecida y reportar tokens y errores léxicos con precisión. |

### 1.3. Objetivo SMART

| SMART                         | Definición                                                 | Objetivo redactado                                                                                                                                                                                                                                                                                  |
|-------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Específico</b><br>(¿Qué?)  | El objetivo es concreto y tangible.                        | Los estudiantes serán capaces de diseñar, implementar y probar un analizador léxico funcional con interfaz gráfica que permita definir tokens mediante expresiones regulares, procesar archivos de código fuente, generar tablas de tokens y detectar errores léxicos.                              |
| <b>Medible</b><br>(¿Cuánto?)  | El objetivo tiene una medida objetiva de éxito.            | Lograr un analizador que reconozca correctamente al menos el 95% de los tokens definidos en los casos de prueba proporcionados, genere reportes de errores precisos con información de línea y columna, y cuente con una interfaz gráfica intuitiva que supere una evaluación de usabilidad básica. |
| <b>Alcanzable</b><br>(¿Cómo?) | El objetivo debe ser posible con los recursos disponibles. | Utilizando lenguajes de programación que soporten el paradigma orientado a objetos, implementación manual del autómata finito determinista (AFD), herramientas de control de versiones para la gestión                                                                                              |

|                              |                                                                               |                                                                                                                                                                                                                         |
|------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              |                                                                               | del código, y software de modelado para la elaboración de diagramas UML y del AFD.                                                                                                                                      |
| <b>Realista (¿Para qué?)</b> | El objetivo contribuye a metas más amplias.                                   | Preparación para cursos avanzados en el área de lenguajes y compiladores, mediante la adquisición de experiencia en el desarrollo de soluciones de software y en la implementación de la primera fase de un compilador. |
| <b>A Tiempo (¿Cuándo?)</b>   | El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso | Completar en 4 semanas: diseño y AFD (semana 1), motor léxico (semana 2), integración de GUI y reportes (semana 3), pruebas y documentación (semana 4).                                                                 |

## 2. Resumen Ejecutivo

Este proyecto pertenece al curso de Lenguajes Formales y de Programación. El problema que aborda es la comprensión práctica de la primera fase de un compilador: la tokenización de código fuente. La solución propuesta consiste en el desarrollo de un analizador léxico que procesa archivos de texto con una sintaxis definida, aplica un autómata finito determinista (AFD) para el reconocimiento de tokens, genera reportes de los elementos identificados y de los errores léxicos, y presenta los resultados mediante una interfaz de usuario.

El proyecto integra competencias en teoría de autómatas, expresiones regulares, estructuras de datos, paradigmas de programación y desarrollo de interfaces. Su alcance se limita al análisis léxico, sentando las bases para proyectos posteriores que incorporen análisis sintáctico.

## 3. Enunciado del Proyecto

### 3.1 Descripción del problema a resolver

En el desarrollo de compiladores e intérpretes, el análisis léxico transforma una secuencia de caracteres en una secuencia de tokens significativos. Para los estudiantes de Ciencias de la Computación, visualizar este proceso mediante una sintaxis familiar facilita el aprendizaje, ya que permite verificar de forma intuitiva la validez de los tokens reconocidos. El problema específico que este proyecto aborda es la ausencia de una herramienta que permita cargar código fuente con una sintaxis simplificada, procesarlo mediante un autómata finito determinista (AFD) implementado manualmente, y generar reportes estructurados de los tokens identificados y los errores léxicos encontrados.

## 3.2 Alcance del proyecto – Sintaxis del Lenguaje

El lenguaje de entrada está basado en un subconjunto de JavaScript. A continuación se define con precisión cada categoría de token que el AFD debe reconocer, los patrones que los describen, y las reglas que los distinguen.

### 3.2.1 Tokens del Lenguaje (basado en JavaScript)

La siguiente tabla define los 10 tipos de tokens que el analizador léxico debe reconocer. Cada token está identificado por un nombre de tipo, su patrón léxico y ejemplos concretos de lexemas válidos.

| # | Tipo de Token       | Patrón / Regla                                                                                                                                                                         | Ejemplos de Lexemas                                                                                                |
|---|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1 | PALABRA_RESERVADA   | Secuencia de letras que coincide exactamente con una palabra clave de la lista reservada. Tiene prioridad sobre IDENTIFICADOR.                                                         | var, let, const, function, return, if, else, while, for, do, break, continue, true, false, null, undefined, typeof |
| 2 | IDENTIFICADOR       | Inicia con letra (a-z, A-Z) o guion bajo (_). Continúa con letras, dígitos (0-9) o guiones bajos. No puede ser palabra reservada.                                                      | miVariable _contador<br>calcularTotal x nombre2<br>__proto__                                                       |
| 3 | ENTERO              | Uno o más dígitos consecutivos (0-9). No puede comenzar con 0 seguido de otro dígito (sin octales). El signo negativo NO forma parte del token; es un operador separado.               | 0 42 100 9999                                                                                                      |
| 4 | DECIMAL             | Parte entera (uno o más dígitos), seguida obligatoriamente de un punto (.), seguida de al menos un dígito. Ejemplo de patrón: [0-9]+\.[0-9]+                                           | 3.14 0.5 100.0 2.71828                                                                                             |
| 5 | CADENA              | Texto delimitado por comillas dobles (") o comillas simples ('). Puede contener cualquier carácter excepto un salto de línea sin escapar. Soporta secuencias de escape: \n \t \\ \" \' | "hola mundo" 'JavaScript'<br>"línea\nnueva" "it's ok"                                                              |
| 6 | OPERADOR_ARITMETICO | Símbolo simple de operación aritmética.                                                                                                                                                | + - * / % ** (potencia)                                                                                            |
| 7 | OPERADOR_RELACIONAL | Comparación entre dos valores. Los de dos                                                                                                                                              | == != === !== < > <= >=                                                                                            |

| #  | Tipo de Token       | Patrón / Regla                                                                                                                                                                | Ejemplos de Lexemas                         |
|----|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
|    |                     | caracteres deben reconocerse como un solo token (no como dos tokens separados).                                                                                               |                                             |
| 8  | OPERADOR_LOGICO     | Operadores lógicos booleanos.                                                                                                                                                 | &&    !                                     |
| 9  | OPERADOR_ASIGNACION | Asignación simple o compuesta.                                                                                                                                                | = += -= *= /= %=                            |
| 10 | SIMBOLO             | Caracteres de puntuación con significado sintáctico. Cada carácter es un token independiente.                                                                                 | ; , . ( ) { } [ ] :                         |
| 11 | COMENTARIO_LINEA    | Inicia con // y abarca hasta el final de la línea (\n o EOF). El analizador lo reconoce como token pero puede omitirlo del reporte de tokens o marcarlo como tipo COMENTARIO. | // esto es un comentario<br>//TODO: revisar |
| 12 | COMENTARIO_BLOQUE   | Inicia con /* y termina con */. Puede abarcar múltiples líneas. Si no se cierra antes del EOF es un error léxico.                                                             | /* comentario */ /*<br>varias\nlíneas */    |

### Reglas de prioridad y manejo especial:

- Palabras reservadas vs. Identificadores: el AFD primero consume la secuencia alfanumérica completa y luego consulta una tabla de palabras reservadas. Si coincide, el token es PALABRA\_RESERVADA; si no, es IDENTIFICADOR.
- Decimal vs. Entero: al leer dígitos, si el siguiente carácter es un punto (.) y el carácter posterior a ese punto es también un dígito, se consume como DECIMAL. Si no, los dígitos son ENTERO y el punto se analiza como SIMBOLO.
- Operadores de dos caracteres: al leer =, !, <, >, +, -, \*, /, % o &, |, el AFD hace lookahead de un carácter para determinar si forma un token de dos caracteres (==, !=, <=, >=, ===, !==, +=, -=, \*=, /=, %=, &&, ||, \*\*).
- Cadenas sin cerrar: si se abre una cadena con " o ' y se alcanza el final de línea o EOF sin encontrar el delimitador de cierre, se emite un ERROR\_LEXICO con el mensaje "Cadena sin cerrar" y la posición de apertura.
- Comentario de bloque sin cerrar: si se abre /\* y se alcanza EOF sin \*/, se emite un ERROR\_LEXICO con el mensaje "Comentario de bloque sin cerrar" y la posición de apertura.
- Caracteres no reconocidos: cualquier carácter que no inicie ningún patrón válido (por ejemplo @, #, \$, ^, ~, `) se reporta como ERROR\_LEXICO con el mensaje "Carácter no reconocido: '<char>'" y su posición exacta.
- Espacios en blanco y saltos de línea: son delimitadores silenciosos. No generan token. El analizador los consume y actualiza el contador de línea/columna.

•

### 3.2.2 Ejemplos de Archivos de Entrada

A continuación se presentan tres archivos de ejemplo que el analizador debe procesar. El Ejemplo A contiene código completamente válido; el Ejemplo B contiene errores léxicos deliberados; el Ejemplo C combina ambos casos.

#### Ejemplo A — Código Válido (sin errores léxicos)

Nombre de archivo sugerido: entrada\_valida.js

```
// Programa de ejemplo: cálculo de factorial
function factorial(n) {
    if (n === 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

var resultado = factorial(5);
var mensaje   = "El resultado es: ";

/* Impresión del resultado */
let total = resultado + 10.5;
let flag  = (total >= 100) && (resultado !== 0);
```

Tokens esperados (resumen por tipo):

| Tipo de Token       | Cantidad aprox. | Algunos lexemas reconocidos                   |
|---------------------|-----------------|-----------------------------------------------|
| PALABRA_RESERVADA   | 10              | function, if, else, return, var, let          |
| IDENTIFICADOR       | 9               | Factorial, n, resultado, mensaje, total, flag |
| ENTERO              | 3               | 0 1 5                                         |
| DECIMAL             | 1               | 10.5                                          |
| CADENA              | 1               | "El resultado es: "                           |
| OPERADOR_RELACIONAL | 3               | === >= !==                                    |
| OPERADOR_ARITMETICO | 3               | * - +                                         |
| OPERADOR_ASIGNACION | 4               | = = = =                                       |
| OPERADOR_LOGICO     | 1               | &&                                            |
| SIMBOLO             | 20              | ( ) { } ; ,                                   |
| COMENTARIO_LINEA    | 1               | // Programa de ejemplo...                     |
| COMENTARIO_BLOQUE   | 1               | /* Impresión del resultado */                 |

| Tipo de Token   | Cantidad aprox. | Algunos lexemas reconocidos |
|-----------------|-----------------|-----------------------------|
| ERRORES LÉXICOS | 0               | —                           |

### Ejemplo B — Código con Errores Léxicos

Nombre de archivo sugerido: entrada\_errores.js

```
var precio = 99.;           // ERROR: decimal incompleto (falta dígito tras el
                             punto)
let nombre = "Hola mundo;  // ERROR: cadena sin cerrar
var total  = precio @ 2;    // ERROR: carácter no reconocido '@'
/* bloque sin cerrar       // ERROR: comentario de bloque sin cerrar
let x = 10;
let y = x + #5;            // ERROR: carácter no reconocido '#'
```

Errores léxicos esperados:

| Línea | Columna | Descripción del Error                                                                            |
|-------|---------|--------------------------------------------------------------------------------------------------|
| 1     | 12      | Decimal incompleto: se esperaba dígito después del punto en '99.'                                |
| 2     | 12      | Cadena sin cerrar: se abrió con '"' en columna 12 y no se encontró cierre antes del fin de línea |
| 3     | 20      | Carácter no reconocido: '@'                                                                      |
| 4     | 1       | Comentario de bloque sin cerrar: se abrió '/*' en línea 4, columna 1 y no se encontró '*'        |
| 6     | 13      | Carácter no reconocido: '#'                                                                      |

### Ejemplo C — Código Mixto (tokens válidos y errores)

Nombre de archivo sugerido: entrada\_mixta.js

```
function suma(a, b) {
    var resultado = a + b;
    let etiqueta = 'Suma: ';    // cadena con comilla simple
    let pi       = 3.14;
    let error1   = "sin cierre; // ERROR léxico línea 5
    return resultado;
}

var r = suma(3, 4.5);
let ok = (r > 0) || (r === 0);
let mal = r ^ 2;               // ERROR: '^' no reconocido
```

Este archivo produce tanto tokens válidos como errores léxicos. El analizador debe continuar procesando después de cada error (modo de recuperación), acumulando todos los errores encontrados y reportándolos al final.

### 3.2.3 Generación de Reportes

El sistema debe generar exactamente dos reportes al finalizar el análisis léxico de un archivo. Ambos reportes deben poder visualizarse en la interfaz gráfica y exportarse a un archivo HTML. Los reportes son independientes entre sí y siempre deben generarse, incluso si no hay errores o si todos los tokens son erróneos.

#### Reporte 1 — Tabla de Tokens Reconocidos

Este reporte lista todos los tokens que el AFD reconoció exitosamente. Los errores léxicos NO aparecen en esta tabla; tienen su propio reporte. Los comentarios pueden incluirse o excluirse según la configuración del analizador (configurable en la GUI).

Contenido obligatorio de cada fila:

| Campo         | Tipo de Dato          | Descripción                                                             |
|---------------|-----------------------|-------------------------------------------------------------------------|
| No.           | Entero (1, 2, 3, ...) | Número de orden del token en la secuencia de entrada.                   |
| Lexema        | Cadena de texto       | El texto exacto leído del archivo fuente (ej. factorial, 3.14, "hola"). |
| Tipo de Token | Enumeración (texto)   | Nombre del tipo: PALABRA_RESERVADA, IDENTIFICADOR, ENTERO, etc.         |
| Línea         | Entero                | Número de línea donde inicia el lexema (contando desde 1).              |
| Columna       | Entero                | Número de columna donde inicia el lexema (contando desde 1).            |

Ejemplo visual del Reporte 1:

| No. | Lexema                    | Tipo de Token     | Línea | Columna |
|-----|---------------------------|-------------------|-------|---------|
| 1   | // Programa de ejemplo... | COMENTARIO_LINEA  | 1     | 1       |
| 2   | function                  | PALABRA_RESERVADA | 2     | 1       |
| 3   | factorial                 | IDENTIFICADOR     | 2     | 10      |
| 4   | (                         | SIMBOLO           | 2     | 19      |
| 5   | n                         | IDENTIFICADOR     | 2     | 20      |
| 6   | )                         | SIMBOLO           | 2     | 21      |
| 7   | {                         | SIMBOLO           | 2     | 23      |
| 8   | if                        | PALABRA_RESERVADA | 3     | 5       |

#### Reporte 2 — Reporte de Errores Léxicos

Este reporte lista todos los errores léxicos encontrados durante el análisis. Si el archivo de entrada no contiene ningún error léxico, el reporte debe generarse de todas formas pero indicando explícitamente que no se encontraron errores.

Regla de recuperación de errores: cuando el AFD encuentra un carácter que no puede procesar, debe registrar el error, consumir ese carácter y continuar el análisis desde el siguiente. Esto garantiza que el reporte acumule todos los errores del archivo, no solo el primero.

Contenido obligatorio de cada fila:

| Campo             | Tipo de Dato          | Descripción                                                                                                |
|-------------------|-----------------------|------------------------------------------------------------------------------------------------------------|
| No.               | Entero (1, 2, 3, ...) | Número de orden del error en la secuencia de análisis.                                                     |
| Carácter / Lexema | Cadena de texto       | El carácter o secuencia que causó el error (ej. @, #, "Hola sin cierre).                                   |
| Tipo de Error     | Enumeración (texto)   | Categoría del error: CARACTER_NO_RECONOCIDO, CADENA_SIN_CERRAR, DECIMAL_INCOMPLETO, COMENTARIO_SIN_CERRAR. |
| Línea             | Entero                | Número de línea donde se detectó el error (contando desde 1).                                              |
| Columna           | Entero                | Número de columna donde inicia el error (contando desde 1).                                                |
| Descripción       | Cadena de texto       | Mensaje explicativo del error, claro y específico para el usuario.                                         |

Categorías de error léxico reconocidas:

| Tipo de Error          | Descripción y mensaje sugerido                                                                                                                                       |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CARACTER_NO_RECONOCIDO | El carácter leído no pertenece al alfabeto del lenguaje. Mensaje: "Carácter no reconocido: '<char>' en línea L, columna C"                                           |
| CADENA_SIN_CERRAR      | Se abrió una cadena con " o ' y se llegó al final de línea o EOF sin encontrar el delimitador de cierre. Mensaje: "Cadena sin cerrar iniciada en línea L, columna C" |
| DECIMAL_INCOMPLETO     | Se leyó un número seguido de punto pero sin dígitos posteriores (ej. 99.). Mensaje: "Decimal incompleto: se esperaba dígito después del punto en línea L, columna C" |
| COMENTARIO_SIN_CERRAR  | Se abrió /* y se llegó al EOF sin encontrar */. Mensaje: "Comentario de bloque sin cerrar, abierto en línea L, columna C"                                            |

Integración de los reportes en la interfaz de usuario:

- Ambos reportes deberán visualizarse en la aplicación mediante secciones diferenciadas, como pestañas o paneles dentro de la ventana principal.

- El primer reporte deberá presentarse en un componente tabular con columnas que incluyan: número, lexema, tipo, línea y columna.
- El segundo reporte deberá mostrarse en un componente tabular con columnas como: número, carácter, tipo de error, línea, columna y descripción.
- Se deberá incluir una opción que permita exportar los reportes generados a archivos en formato estructurado, almacenados en la misma ubicación del archivo analizado.
- La interfaz deberá proporcionar retroalimentación al usuario al finalizar el proceso, mostrando un resumen del análisis realizado, incluyendo la cantidad de tokens reconocidos y errores léxicos detectados.

### Requerimientos Técnicos

| Componente                   | Especificación                                                                                                                                                                                                                          |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lenguaje de programación     | Lenguaje de programación de propósito general compatible con el desarrollo de aplicaciones de software.                                                                                                                                 |
| Librería de interfaz gráfica | Herramientas o bibliotecas que permitan el desarrollo de interfaces gráficas o entornos visuales interactivos.                                                                                                                          |
| Paradigma                    | Aplicación del paradigma de programación orientada a objetos, estructurando la solución en componentes como analizador léxico, representación de tokens, autómata, manejo de errores y generación de reportes.                          |
| Implementación del AFD       | Implementación manual del autómata finito determinista, sin depender exclusivamente de utilidades automatizadas de reconocimiento de patrones; los estados podrán representarse mediante funciones, métodos o estructuras equivalentes. |
| Control de versiones         | Git + GitHub (privado). Commits descriptivos por funcionalidad. README completo con instrucciones de compilación.                                                                                                                       |
| Compilación                  | Uso de herramientas de construcción o entornos de ejecución compatibles con distintos sistemas operativos.                                                                                                                              |
| Reportes                     | Generación de reportes en formatos estructurados y su visualización dentro de la interfaz de usuario mediante componentes tabulares u otros mecanismos adecuados.                                                                       |

### 3.3 Entregables

| Entregable              | Descripción                                                                                                                                                                                                                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Código Fuente           | Implementación completa del sistema organizada en módulos o componentes, debidamente documentada y estructurada. Incluye el autómata finito determinista, el motor de análisis léxico, la generación de reportes y la interfaz de usuario. Entregado mediante un repositorio de control de versiones. |
| Manual Técnico (PDF)    | Documento que describa la arquitectura del sistema, diagramas de modelado (como diagramas de clases y secuencia), representación del AFD con su tabla de transiciones, descripción de los reportes generados y justificación de las decisiones de diseño.                                             |
| Manual de Usuario (PDF) | Documento que incluya instrucciones paso a paso para la ejecución del sistema, carga de archivos, interpretación de resultados y uso de las funcionalidades principales, apoyado con ejemplos o capturas ilustrativas.                                                                                |
| Casos de Prueba (PDF)   | Conjunto documentado de casos de prueba que incluyan diferentes escenarios (válidos, con errores y mixtos), especificando entradas, resultados esperados y resultados obtenidos para cada uno de los reportes generados.                                                                              |
| Repositorio GitHub      | Espacio de almacenamiento del proyecto con control de versiones, historial de cambios documentado, configuración adecuada para el manejo de archivos y documentación con instrucciones de uso y dependencias del sistema.                                                                             |

## 4. Material de apoyo

- Compilers: Principles, Techniques, and Tools — Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, Monica S. Lam. Capítulos relacionados con análisis léxico y fundamentos de compiladores.
- Introduction to Automata Theory, Languages, and Computation — John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman.
- Documentación oficial de herramientas de desarrollo de interfaces gráficas y entornos de programación utilizados en el proyecto.
- Especificaciones formales de lenguajes de programación o gramáticas léxicas utilizadas como referencia para la definición de la sintaxis de entrada.
- Material académico del curso relacionado con teoría de autómatas, lenguajes formales y análisis léxico.
- Repositorios o ejemplos proporcionados por el equipo docente como guía de implementación.

**Fase 1 — Investigación y Diseño (Semana 1)**

- Definir la lista final de tokens y palabras reservadas del subconjunto JavaScript a soportar.
- Diseñar el AFD completo: diagrama de estados y tabla de transiciones para todos los tokens.
- Diseñar la arquitectura UML: clases Lexer, Token, AFD, ErrorHandler, ReportGenerator, MainWindow.

**Fase 2 — Implementación del Motor Léxico (Semana 2)**

- Definir la estructura de representación de tokens (lexema, tipo, línea, columna).
- Implementar el AFD de forma manual utilizando un enfoque estructurado u orientado a objetos.
- Desarrollar el componente de análisis léxico: lectura secuencial de caracteres, reconocimiento de tokens y registro de errores.
- Implementar el módulo de manejo de errores léxicos, organizados por categorías.
- Realizar pruebas iniciales del motor léxico de forma independiente antes de su integración con la interfaz de usuario.

**Fase 3 — Integración de GUI y Generación de Reportes (Semana 3)**

- Integrar Lexer con la GUI: botón «Cargar Archivo», botón «Analizar», botón «Exportar Reportes».
- Implementar ReportGenerator: generar los dos archivos HTML con el formato especificado en 3.2.3.
- Agregar la barra de estado con el resumen de tokens y errores al finalizar el análisis.

**Fase 4 — Pruebas y Documentación (Semana 4)**

- Ejecutar los cinco casos de prueba mínimos y documentar resultados obtenidos.
- Redactar Manual Técnico con diagramas UML finales y diagrama del AFD.
- Redactar Manual de Usuario con capturas de pantalla de la aplicación y los reportes generados.
- Preparar la presentación oral con demostración en vivo de los tres archivos de ejemplo.

## 6. Recursos y herramientas a utilizar

| Categoría            | Herramienta / Recurso                                                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------|
| IDE                  | Entorno de desarrollo integrado (IDE) compatible con el lenguaje seleccionado y de acceso libre o institucional. |
| Compilador           | Herramientas de compilación o ejecución compatibles con el lenguaje utilizado y el sistema operativo disponible. |
| Framework GUI        | Bibliotecas o frameworks que permitan el desarrollo de interfaces gráficas o entornos visuales interactivos.     |
| Control de versiones | Git + GitHub (repositorio privado)                                                                               |
| Diagramas UML/AFD    | Draw.io (gratuito, online) o Lucidchart                                                                          |

| Categoría          | Herramienta / Recurso                                          |
|--------------------|----------------------------------------------------------------|
| Documentación      | Word/LibreOffice Writer para manuales exportados a PDF         |
| Archivos de prueba | Los tres archivos .js de ejemplo definidos en la sección 3.2.2 |

## 7. Desarrollo de Habilidades Blandas

### Autogestión del Tiempo

Se recomienda dedicar entre 5 y 6 horas semanales, siguiendo las cuatro fases del cronograma. El estudiante debe crear un plan personal con hitos verificables al cierre de cada semana.

### Responsabilidad

Cada estudiante es responsable de la autoría íntegra de su código, documentación y casos de prueba. Todo trabajo será verificado con herramientas de detección de autoría.

### Resolución de Problemas

Se esperan desafíos técnicos en la implementación del AFD (gestión de lookahead, recuperación de errores) y en la integración GUI (señales/slots, población de tablas). Se fomenta la consulta de documentación oficial antes de solicitar ayuda.

### Pensamiento Crítico

El estudiante debe justificar sus decisiones de diseño del AFD y la estructura de clases en el Manual Técnico, evaluando alternativas y fundamentando la solución elegida.

## 8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

| Tipo                   | Fecha Inicio | Fecha Fin |
|------------------------|--------------|-----------|
| Asignación de Proyecto | xx/xx/xxxx   | xx/xx/xxx |

## 9. Rúbrica de Calificación

| Requisito                 | Descripción                                                                                                                                                                               | Cumple (Sí/No) |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Tecnología establecida    | Uso de tecnologías de desarrollo compatibles con la implementación de aplicaciones de software y la construcción de interfaces de usuario.                                                |                |
| Herramientas requeridas   | Uso de un sistema de control de versiones con historial de cambios documentado y repositorio estructurado con información del proyecto..                                                  |                |
| Documentación obligatoria | Elaboración de manual técnico, manual de usuario y documentación de casos de prueba en formatos estándar.                                                                                 |                |
| Funcionalidad mínima      | Capacidad de cargar archivos de entrada con una sintaxis definida, reconocer múltiples tipos de tokens mediante un AFD, y generar reportes de tokens y errores con los campos requeridos. |                |

### 9.1 Requisitos para Optar a la Calificación

| No.                              | Criterio                               | Pts | Satisfactorio (100%–61%)                                                                                                                                            | Necesita Mejorar (60%–0%)                                                                             | Pts Obtenidos |
|----------------------------------|----------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|---------------|
| <b>HABILIDADES — 40 puntos</b>   |                                        |     |                                                                                                                                                                     |                                                                                                       |               |
| 1.1                              | Documentación Técnica (Manual Técnico) | 10  | Completo con diagramas UML, diagrama del AFD con tabla de transiciones, descripción de las clases y justificación de decisiones. Visualmente profesional.           | Sin diagramas, sin descripción del AFD, algoritmos sin explicar o presentación desorganizada.         |               |
| 1.2                              | Manual de Usuario                      | 8   | Completo con instrucciones de compilación, capturas de los dos reportes generados, explicación de carga de archivos y exportación HTML.                             | Sin capturas de los reportes, instrucciones confusas o ausentes.                                      |               |
| 1.3                              | Repositorio GitHub                     | 7   | Commits descriptivos por funcionalidad, .gitignore y README con dependencias e instrucciones, incluye archivos de ejemplo JS.                                       | Pocos commits genéricos, README vacío o incompleto, sin archivos de prueba.                           |               |
| 1.4                              | Presentación                           | 8   | Cubre introducción, demostración en vivo de los tres archivos de ejemplo, ambos reportes exportados y explicación del AFD implementado.                             | Sin demostración funcional, sin mostrar los reportes o sin explicación técnica del AFD.               |               |
| 1.5                              | Casos de Prueba y Testing              | 7   | Mínimo 5 casos (válido, errores, mixto + casos borde). Cada caso documenta los dos reportes generados: tabla de tokens y tabla de errores con resultados obtenidos. | Menos de 5 casos, solo documenta tokens sin documentar errores, o sin comparar esperado vs. obtenido. |               |
| <b>CONOCIMIENTOS — 60 puntos</b> |                                        |     |                                                                                                                                                                     |                                                                                                       |               |

| No.  | Criterio                              | Pts | Satisfactorio (100%–61%)                                                                                                                                                                                                                    | Necesita Mejorar (60%–0%)                                                                                                       | Pts Obtenidos |
|------|---------------------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------|
| 2.1  | Carga y Procesamiento de Archivos     | 8   | Carga correctamente archivos .js, valida existencia y legibilidad, actualiza contadores de línea/columna con precisión al encontrar \n y \t.                                                                                                | No carga archivos o los contadores de línea/columna son incorrectos frecuentemente.                                             |               |
| 2.2  | Reconocimiento de Tokens mediante AFD | 15  | AFD implementado manualmente con POO reconoce los 12 tipos de token al 95–100% en los casos de prueba. Lookahead correcto para operadores y literales compuestos.                                                                           | Reconoce menos del 80% de los tokens, no distingue operadores compuestos (== de =) o no implementa lookahead.                   |               |
| 2.3  | Reporte 1 — Tabla de Tokens           | 12  | Genera correctamente los campos No., Lexema, Tipo, Línea y Columna para todos los tokens.                                                                                                                                                   | Faltan campos, posiciones incorrectas frecuentemente, no exporta a HTML o el HTML no sigue el formato especificado.             |               |
| 2.4  | Reporte 2 — Errores Léxicos           | 13  | Detecta las cuatro categorías de error (carácter no reconocido, cadena sin cerrar, decimal incompleto, comentario sin cerrar). Reporta posición exacta y mensaje descriptivo. Recuperación: continúa tras cada error. Genera HTML correcto. | No detecta alguna categoría de error, posiciones incorrectas, no se recupera tras el primer error o no exporta el reporte HTML. |               |
| 2.5  | Interfaz Gráfica                      | 9   | GUI con Cargar/Analizar/Exportar funcionales, barra de estado con resumen. Diseño profesional e intuitivo.                                                                                                                                  | Faltan pestañas o botones, la interfaz no muestra alguno de los dos reportes, barra de estado ausente o diseño confuso.         |               |
| 2.6* | Funcionalidades Opcionales            | 3   | Implementa al menos una: resaltado de sintaxis en el área                                                                                                                                                                                   | No implementa funcionalidades opcionales o las implementadas están incompletas.                                                 |               |

| No.   | Criterio | Pts | Satisfactorio (100%–61%)                                                                                                        | Necesita Mejorar (60%–0%) | Pts Obtenidos |
|-------|----------|-----|---------------------------------------------------------------------------------------------------------------------------------|---------------------------|---------------|
|       |          |     | de código, visualización gráfica del AFD, estadísticas (frecuencia por tipo de token) o exportación en formato adicional (CSV). |                           |               |
| TOTAL |          | 100 |                                                                                                                                 |                           |               |